Compiler Construction Project

Phase II

German Language Compiler

Submitted by: Syed Mohammad Hussain Bukhari

Submitted to: Miss Aneela Mehmood

CFG:

Non-Terminals:

1. program
2. stml_list
3. stmt
4. declearation_stmt
5. assignment_stmt
6. conditional_stmt
7. elseif_chain
8. optional_else
9. loop_stmt
10. output_stmt
11. expr

Terminals:

1. keywords => wenn, sonst, sonst wenn, fur, für, solange, klasse, funktion, drucke, eingabe, gebe, importiere, definiere, wahle, wähle, fall, abbrechen, wahr, falsch
2. punctuations => lbrace ({), rbrace (}), lparen (), rparen ()), colon (:), semicolon (;)
3. identifiers/literals => id, string, number
4. operators => eq (==), ne (!=), le (<=), ge (>=), plusplus (++), minusminus (--), pluseq (+=), minuseq (-=), timeseq (*=), diveq (/=), plus (+), minus (-), mult (*), div (/), mod (%), assign (=), lt (<), gt (>)

CFG:

	Non-Terminal	Terminals
1.	program:	stmt_list
2.	stmt_list:	stmt stmt_list | stmt
3.	stmt:	declearation_stmt | conditional_stmt | loop_stmt | output_stmt
4.	declearation_stmt:	FUNKTION ID LPAREN RPAREN LBRACE stmt_list RBRACE | KLASSE ID LBRACE stmt_list RBRACE

5. assignment_stmt: ID ASSIGN expr SEMICOLON | ID PLUSEQ expr SEMICOLON | ID MINUSEQ expr SEMICOLON | ID TIMESEQ expr SEMICOLON | ID DIVEQ expr SEMICOLON
6. conditional_stmt: WENN LPAREN expr RPAREN LBRACE stmt_list RBRACE elseif_chain optional_else
7. elseif_chain: SONSTWENN LPAREN expr RPAREN LBRACE stmt_list RBRACE elseif_chain | ε
8. optional_else: SONST LBRACE stmt_list RBRACE | ε
9. loop_stmt: SOLANGE LPAREN expr RPAREN LBRACE stmt_list RBRACE | FUR LPAREN assignment_stmt expr SEMICOLON assignment_stmt RPAREN LBRACE stmt_list RBRACE
10. output_stmt: DRUCKE LPAREN expr RPAREN SEMICOLON | EINGABE LPAREN ID RPAREN SEMICOLON
11. expr: NUMBER | STRING | ID | expr PLUS expr | expr MINUS expr | expr MULT expr | expr DIV expr | expr MOD expr | expr EQ expr | expr NE expr | expr GT expr | expr LT expr | LPAREN expr RPAREN

First and Follow set:

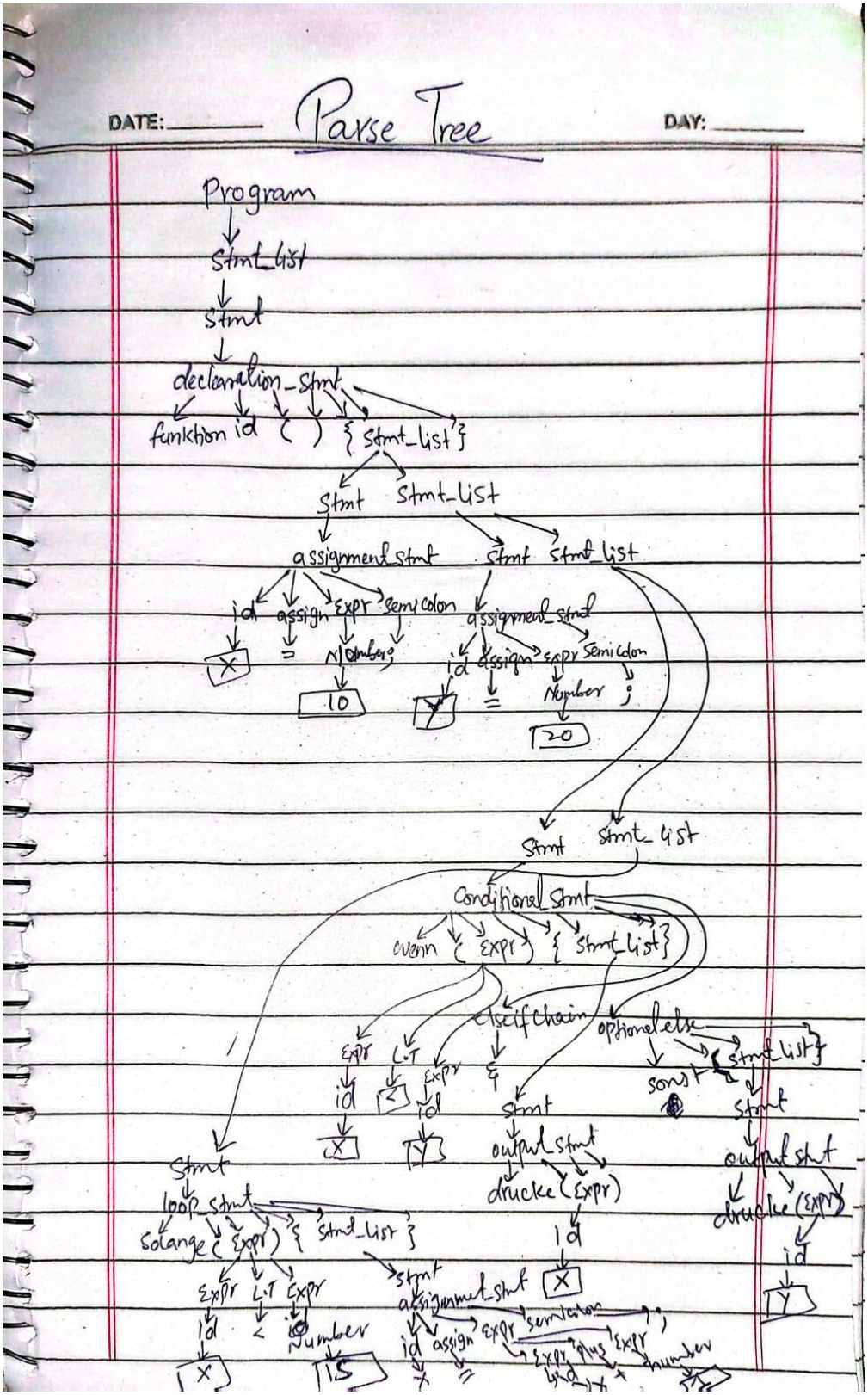
Non-terminal	Rules	First	Follow
stmt	declearation_stmt | conditional_stmt | loop_stmt | output_stmt	{ FUNKTION, KLASSE, ID, WENN, SOLANGE, FUR, DRUCKE, EINGABE }	{ FUNKTION, KLASSE, ID, WENN, SOLANGE, FUR, DRUCKE, EINGABE, RBRACE, $ }
Expr	NUMBER | STRING | ID | expr PLUS expr | expr MINUS expr | expr MULT expr | expr DIV expr | expr MOD expr | expr EQ expr | expr NE expr | expr GT expr | expr LT expr | LPAREN expr RPAREN	{ NUMBER, STRING, ID }	{ PLUS, MINUS, MULT, DIV, MOD, EQ, NE, LT, GT, RPAREN, SEMICOLON }

Test Program:

funktion myFunc() {

```
    x = 10;
    y = 20;
    wenn (x < y) {
        drucke(x);
    } sonst {
        drucke(y);
    }
    solange (x < 15) {
        x = x + 1;
    }
}
```

Parse Tree:

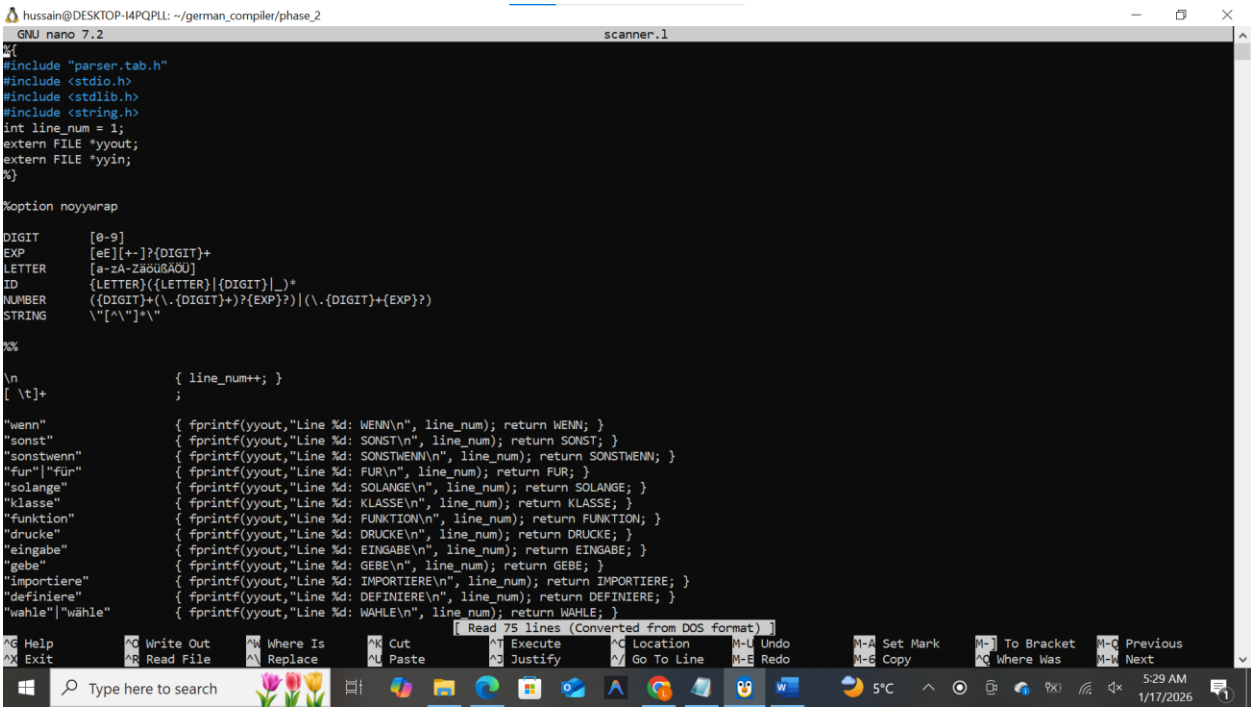

DATE:
Parse Tree
DAY:
Program
stmt_list
stmt
declaration_stmt
funktion id ( ) { stmt_list }
stmt stmt_list
assignment stmt stmt stmt_list
id assign expr semicolon
X = Number ;
10
id assign expr semicolon
Y = Number ;
20
stmt stmt_list
Conditional stmt
ovenn (expr) { stmt_list }
elseif chain optional else
expr L.T expr
id < id
X Y
stmt
output stmt
drucke (expr)
id
X
stmt
loop stmt
solange (expr) { stmt_list }
expr L.T expr
id < Number
X 15
stmt
assignment stmt
id assign expr semicolon
expr plus expr
number
optional else
const { stmt_list }
stmt
output stmt
drucke (expr)
id
Y

Explanation of Phase 1 tokens usage

- Tokens like ID, NUMBER, STRING, WENN, SONST, FUR, etc. are directly reused from Phase 1 scanner.
- Scanner.l file from phase 1 is modified because initially it was just printing tokens but it is also returning tokens too,
- All operators and punctuations (+, -, =, {, }) are used exactly as in Phase 1.
- Parser does not redefine any tokens, ensuring Phase 1 and Phase 2 are consistent.

Code and Screenshots:

Scanner.l



hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2

GNU nano 7.2 scanner.l

```
"fall"              { fprintf(yyout,"Line %d: FALL\n", line_num); return FALL; }
"abbrechen"         { fprintf(yyout,"Line %d: ABBRECHEN\n", line_num); return ABBRECHEN; }
"wahr"              { fprintf(yyout,"Line %d: WAHR\n", line_num); return WAHR; }
"falsch"            { fprintf(yyout,"Line %d: FALSCH\n", line_num); return FALSCH; }

"=="                { fprintf(yyout,"Line %d: EQ\n", line_num); return EQ; }
"!="                { fprintf(yyout,"Line %d: NE\n", line_num); return NE; }
"<="                { fprintf(yyout,"Line %d: LE\n", line_num); return LE; }
">="                { fprintf(yyout,"Line %d: GE\n", line_num); return GE; }
"++"                { fprintf(yyout,"Line %d: PLUSPLUS\n", line_num); return PLUSPLUS; }
"--"                { fprintf(yyout,"Line %d: MINUSMINUS\n", line_num); return MINUSMINUS; }
"+="                { fprintf(yyout,"Line %d: PLUSEQ\n", line_num); return PLUSEQ; }
"-="                { fprintf(yyout,"Line %d: MINUSEQ\n", line_num); return MINUSEQ; }
"*="                { fprintf(yyout,"Line %d: TIMESEQ\n", line_num); return TIMESEQ; }
"/="                { fprintf(yyout,"Line %d: DIVEQ\n", line_num); return DIVEQ; }
"+"                 { fprintf(yyout,"Line %d: PLUS\n", line_num); return PLUS; }
"-"                 { fprintf(yyout,"Line %d: MINUS\n", line_num); return MINUS; }
"*"                 { fprintf(yyout,"Line %d: MULT\n", line_num); return MULT; }
"/"                 { fprintf(yyout,"Line %d: DIV\n", line_num); return DIV; }
"%"                 { fprintf(yyout,"Line %d: MOD\n", line_num); return MOD; }
"="                 { fprintf(yyout,"Line %d: ASSIGN\n", line_num); return ASSIGN; }
"<"                 { fprintf(yyout,"Line %d: LT\n", line_num); return LT; }
">"                 { fprintf(yyout,"Line %d: GT\n", line_num); return GT; }

"{"                 { fprintf(yyout,"Line %d: LBRACE\n", line_num); return LBRACE; }
"}"                 { fprintf(yyout,"Line %d: RBRACE\n", line_num); return RBRACE; }
"("                 { fprintf(yyout,"Line %d: LPAREN\n", line_num); return LPAREN; }
")"                 { fprintf(yyout,"Line %d: RPAREN\n", line_num); return RPAREN; }
":"                 { fprintf(yyout,"Line %d: COLON\n", line_num); return COLON; }
";"                 { fprintf(yyout,"Line %d: SEMICOLON\n", line_num); return SEMICOLON; }

{ID}       { yylval.id = strdup(yytext); fprintf(yyout,"Line %d: ID -> %s\n", line_num, yytext); return ID; }
{NUMBER}   { yylval.num = atof(yytext); fprintf(yyout,"Line %d: NUMBER -> %s\n", line_num, yytext); return NUMBER; }
{STRING}   { yylval.str = strdup(yytext); fprintf(yyout,"Line %d: STRING -> %s\n", line_num, yytext); return STRING; }

.          { fprintf(stderr,"Line %d: ERROR -> %s\n", line_num, yytext); }
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next

Type here to search 5°C 5:30 AM 1/17/2026

Parser.y

hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2

GNU nano 7.2 parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern int line_num;
extern int yylex();
void yyerror(const char *s);
%}

%union {
    char *id;
    double num;
    char *str;
}

%token <id> ID
%token <num> NUMBER
%token <str> STRING
%token WENN SONST SONSTWENN FUR FUER SOLANGE KLASSE FUNKTION
%token DRUCKE EINGABE GEBE IMPORTIERE DEFINIERE WAHLE FALL ABBRECHEN WAHR FALSCH
%token EQ NE LE GE PLUSPLUS MINUSMINUS PLUSEQ MINUSEQ TIMESEQ DIVEQ
%token PLUS MINUS MULT DIV MOD ASSIGN LT GT
%token LBRACE RBRACE LPAREN RPAREN COLON SEMICOLON

%left PLUS MINUS
%left MULT DIV MOD
%left EQ NE LT GT

%%

program : stmt_list
        { printf("Syntax analysis successful\n"); }
        ;

stmt_list : stmt_list stmt
          | stmt
```

[Read 109 lines (Converted from DOS format)]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next

Type here to search 5°C 5:31 AM 1/17/2026

hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2

```
  GNU nano 7.2                                                    parser.y
stmt : declaration_stmt
     | assignment_stmt
     | conditional_stmt
     | loop_stmt
     | output_stmt
     ;

declaration_stmt
    : FUNKTION ID LPAREN RPAREN LBRACE stmt_list RBRACE
    | KLASSE ID LBRACE stmt_list RBRACE
    ;

assignment_stmt
    : ID ASSIGN expr SEMICOLON
    | ID PLUSEQ expr SEMICOLON
    | ID MINUSEQ expr SEMICOLON
    | ID TIMESEQ expr SEMICOLON
    | ID DIVEQ expr SEMICOLON
    ;

conditional_stmt
    : WENN LPAREN expr RPAREN LBRACE stmt_list RBRACE elseif_chain optional_else
    ;

elseif_chain
    : SONSTWENN LPAREN expr RPAREN LBRACE stmt_list RBRACE elseif_chain
    | /* empty */
    ;

optional_else
    : SONST LBRACE stmt_list RBRACE
    | /* empty */
    ;

loop_stmt
    : SOLANGE LPAREN expr RPAREN LBRACE stmt_list RBRACE
    | FUR LPAREN assignment_stmt expr SEMICOLON assignment_stmt RPAREN LBRACE stmt_list RBRACE

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next
```

Type here to search 5°C 5:31 AM 1/17/2026

hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2

```
  GNU nano 7.2                                                    parser.y
loop_stmt
    : SOLANGE LPAREN expr RPAREN LBRACE stmt_list RBRACE
    | FUR LPAREN assignment_stmt expr SEMICOLON assignment_stmt RPAREN LBRACE stmt_list RBRACE
    ;

output_stmt
    : DRUCKE LPAREN expr RPAREN SEMICOLON
    | EINGABE LPAREN ID RPAREN SEMICOLON
    ;

expr : NUMBER
     | STRING
     | ID
     | expr PLUS expr
     | expr MINUS expr
     | expr MULT expr
     | expr DIV expr
     | expr MOD expr
     | expr EQ expr
     | expr NE expr
     | expr LT expr
     | expr GT expr
     | LPAREN expr RPAREN
     ;

%%

void yyerror(const char *s) {
    printf("Syntax error at line %d: %s\n", line_num, s);
}

int main() {
    printf("Parsing started...\n");
    yyparse();
    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next
```

Type here to search 5°C 5:31 AM 1/17/2026

Valid Program tokens:

```
hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2
  GNU nano 7.2                                   tokens.txt
Parsing started...
Line 1: FUNKTION
Line 1: ID -> myFunc
Line 1: LPAREN
Line 1: RPAREN
Line 1: LBRACE
Line 2: ID -> x
Line 2: ASSIGN
Line 2: NUMBER -> 10
Line 2: SEMICOLON
Line 3: ID -> y
Line 3: ASSIGN
Line 3: NUMBER -> 20
Line 3: SEMICOLON
Line 4: WENN
Line 4: LPAREN
Line 4: ID -> x
Line 4: LT
Line 4: ID -> y
Line 4: RPAREN
Line 4: LBRACE
Line 5: DRUCKE
Line 5: LPAREN
Line 5: ID -> x
Line 5: RPAREN
Line 5: SEMICOLON
Line 6: RBRACE
Line 6: SONST
Line 6: LBRACE
Line 7: DRUCKE
Line 7: LPAREN
Line 7: ID -> y
Line 7: RPAREN
Line 7: SEMICOLON
Line 8: RBRACE
Line 9: SOLANGE
Line 9: LPAREN
                                   [ Read 51 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket   M-Q Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was     M-W Next
```

```
hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2
  GNU nano 7.2                                   tokens.txt
Line 4: LPAREN
Line 4: ID -> x
Line 4: LT
Line 4: ID -> y
Line 4: RPAREN
Line 4: LBRACE
Line 5: DRUCKE
Line 5: LPAREN
Line 5: ID -> x
Line 5: RPAREN
Line 5: SEMICOLON
Line 6: RBRACE
Line 6: SONST
Line 6: LBRACE
Line 7: DRUCKE
Line 7: LPAREN
Line 7: ID -> y
Line 7: RPAREN
Line 7: SEMICOLON
Line 8: RBRACE
Line 9: SOLANGE
Line 9: LPAREN
Line 9: ID -> x
Line 9: LT
Line 9: NUMBER -> 15
Line 9: RPAREN
Line 9: LBRACE
Line 10: ID -> x
Line 10: ASSIGN
Line 10: ID -> x
Line 10: PLUS
Line 10: NUMBER -> 1
Line 10: SEMICOLON
Line 11: RBRACE
Line 12: RBRACE
Syntax analysis successful
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket   M-Q Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was     M-W Next
```

Invalid Program tokens:

```
hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2
  GNU nano 7.2                                  tokens1.txt
Parsing started...
Line 1: FUNKTION
Line 1: ID -> myFunc
Line 1: LPAREN
Line 1: RPAREN
Line 1: LBRACE
Line 2: ID -> x
Line 2: ASSIGN
Line 2: NUMBER -> 10
Line 3: WENN
Syntax error at line 3: syntax error

                                [ Read 11 lines ]
^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next
```

```
hussain@DESKTOP-I4PQPLL: ~/german_compiler/phase_2
hussain@DESKTOP-I4PQPLL:~$ cd german_compiler
hussain@DESKTOP-I4PQPLL:~/german_compiler$ cd phase_@
-bash: cd: phase_@: No such file or directory
hussain@DESKTOP-I4PQPLL:~/german_compiler$ cd phase_2
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ touch valid_test_prog.gc
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ touch invalid_test_prog.gc
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ bison -d parser.y
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ flex scanner.l
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ gcc lex.yy.c parser.tab.c -o parser
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ ./parser < valid_test_prog.gc > tokens.txt 2> errorlog.txt
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$ ./parser < invalid_test_prog.gc > tokens1.txt 2> errorlog1.txt
hussain@DESKTOP-I4PQPLL:~/german_compiler/phase_2$
```